

useState() Hooks - Interview Preparation

Modern React 18/19 | Senior Frontend Interview Guide

1. Fifteen Interview Questions

Basic 1. What is useState?

Answer: A Hook that preserves local component state across renders and schedules a new render when it changes. Why: It makes function components interactive without classes. Follow-up: Does a setter mutate state immediately? Common incorrect answer: "It directly changes a variable." Senior-level answer: State is a render snapshot tied to a component instance; a setter queues a future update and React later renders and commits the result.

Basic 2. What does useState return?

Answer: A current value and a setter: `const [isOpen, setIsOpen] = useState(false)`. Why: The pair makes ownership explicit. Follow-up: Is the setter stable? Common incorrect answer: "It returns an object and a mutator." Senior-level answer: Setter identity is stable between renders, so it normally does not cause dependency changes by itself.

Basic 3. Why call Hooks at the top level?

Answer: React associates Hooks by call order. Why: Conditional calls can shift that order and attach state incorrectly. Follow-up: Can a custom Hook be conditional? Common incorrect answer: "It is a lint preference." Senior-level answer: It is a correctness rule that applies equally to custom Hooks.

Basic 4. How is state initialized?

Answer: `useState(initialValue)` uses the initializer on mount. Why: React preserves later state across ordinary renders. Follow-up: When use lazy initialization? Common incorrect answer: "The initializer runs every render." Senior-level answer: Use `useState(() => expensiveSetup())` when initial computation is costly.

Basic 5. Does useState merge objects?

Answer: No; the setter replaces the whole value. Why: Hooks do not use class `setState` shallow merging. Follow-up: How do you update one field? Common incorrect answer: "Missing fields are retained." Senior-level answer: Return a new object: `setUser(prev => ({ ...prev, name: "Asha" })).`

Intermediate 6. Why use a functional updater?

Answer: Use `setCount(prev => prev + 1)` when next state depends on previous state. Why: It composes correctly with queued updates. Follow-up: What do three direct increments do? Common incorrect answer: "Both forms are always identical." Senior-level answer: Direct values read the current render snapshot; functional updaters receive the latest queued prior value.

Intermediate 7. What is state as a snapshot?

Answer: Each render sees a fixed state value; its event handlers close over that value. Why: It explains stale reads in async callbacks. Follow-up: How do you avoid stale values? Common incorrect answer: "React mutates state randomly later." Senior-level answer: Use functional updates, correct effect dependencies, cancellation, or a ref only for non-render mutable values.

Intermediate 8. How does React bail out state work?

Answer: React compares state using `Object.is`; equal values can allow a bailout. Why: It avoids some redundant work. Follow-up: Does React deep compare objects? Common incorrect answer: "Yes, React checks every field." Senior-level answer: It does not deep compare; objects must be updated immutably with a new reference.

Intermediate 9. When should state be lifted up?

Answer: When multiple components need one source of truth or coordinated control. Why: Shared ownership prevents divergence. Follow-up: What is the downside? Common incorrect answer: "All state should be global." Senior-level answer: Lift only to the lowest common owner; excessive lifting broadens rerenders and coupling.

Intermediate 10. When should you avoid useState?

Answer: Avoid it for derived values, complex workflows, remote cache data, and non-UI mutable values. Why: Each has a better model. Follow-up: What alternatives fit? Common incorrect answer: "Every changing value belongs in state." Senior-level answer: Derive values in render, useReducer for workflows, server-state tools for API data, and useRef for non-render values.

Advanced 11. What is automatic batching in React 18?

Answer: React groups many updates from events, promises, timers, and native events into fewer renders. Why: It avoids intermediate commits. Follow-up: Does batching make updates synchronous? Common incorrect answer: "All setters immediately

run together.” Senior-level answer: Batching is scheduling optimization; functional updates remain important for prior-state transitions.

Advanced 12. How does useState work with concurrent rendering?

Answer: React can start, pause, resume, or discard render work before commit. Why: It prioritizes urgent updates such as typing. Follow-up: What is unsafe in render? Common incorrect answer: “Every render reaches the DOM.” Senior-level answer: Render must be pure; use effects or event handlers for external side effects.

Advanced 13. What is Fiber’s relevance?

Answer: Fiber is React’s internal work architecture for scheduling and reconciliation. Why: It supports prioritization and interruptible work. Follow-up: Should app code use lanes? Common incorrect answer: “Fiber is public state API.” Senior-level answer: Lanes and queues are implementation details; rely on React’s public guarantees.

Advanced 14. How do you optimize a large list controlled by state?

Answer: Keep state local, virtualize rows, derive values, and profile before memoizing. Why: Rendering many rows is usually the real cost. Follow-up: Is useMemo always useful? Common incorrect answer: “Memoize everything.” Senior-level answer: Start with ownership and virtualization; memoize measured expensive work and stabilize props only when it helps.

Advanced 15. How do keys affect useState?

Answer: State is retained by component identity; changing key creates a new instance and resets state. Why: Keys participate in reconciliation identity. Follow-up: When is it useful? Common incorrect answer: “Keys are only for list warnings.” Senior-level answer: Use a key intentionally to reset an entity-specific form, never as a casual workaround for wrong state ownership.

2. Five Output-Based Questions

1. Repeated direct update

```
function Counter() {
  const [count, setCount] = useState(0);
  function handleClick() {
    setCount(count + 1);
    setCount(count + 1);
    setCount(count + 1);
  }
  return <button onClick={handleClick}>{count}</button>;
}
```

Expected output: After one click: 1. Explanation: Every expression reads the same snapshot. Interview concept: Batching and snapshots.

2. Functional update

```
function Counter() {
  const [count, setCount] = useState(0);
  function handleClick() {
    setCount(value => value + 1);
    setCount(value => value + 1);
    setCount(value => value + 1);
  }
  return <button onClick={handleClick}>{count}</button>;
}
```

Expected output: After one click: 3. Explanation: Each updater receives the latest queued result. Interview concept: Functional update composition.

3. Object replacement

```
function Profile() {
  const [user, setUser] = useState({ name: "Asha", role: "Admin" });
  function updateName() { setUser({ name: "Ravi" }); }
  return <p>{user.name} - {user.role}</p>;
}
```

Expected output: Ravi - . Explanation: role is removed because setters replace state. Interview concept: No shallow merge.

4. Lazy initializer

```
function Items() {
  const [items] = useState(() => {
    console.log("Creating items");
    return [1, 2, 3];
  });
  return <p>{items.join(", ")}</p>;
}
```

Expected output: 1, 2, 3; the log runs on mount. Explanation: The initialization function avoids repeated computation. Interview concept: Lazy initialization.

5. Mutable Set mistake

```
function Tags() {
  const [selectedIds, setSelectedIds] = useState(new Set());
  function selectTag(id) {
    selectedIds.add(id);
    setSelectedIds(selectedIds);
  }
  return <p>{selectedIds.size}</p>;
}
```

Expected output: Not reliably updated. Explanation: Existing state is mutated and returned with the same reference. Interview concept: Immutable updates. Correct approach: clone first with `new Set(previousIds)`.

3. Five Scenario-Based Questions

Search with 50,000 local records

What would you do? Keep input state urgent, virtualize results, profile filtering and rows, and use memoization only for measured work. For server search, debounce, cancel stale requests, and cache results. Trade-offs: Debouncing delays results; virtualization adds implementation complexity. Performance: `startTransition` or `useDeferredValue` can defer expensive result rendering. Scalability: Put shareable filters in URL state.

Checkout quantity and inventory

What would you do? Keep selected quantity local and treat inventory as server state. Validate again during cart/checkout mutation. Trade-offs: Optimistic UI requires rollback. Performance: Do not write the API on every click without a deliberate strategy. Scalability: Use server-state mutations and invalidation for all cart consumers.

CRM form with dependent fields

What would you do? Use a form library or `useReducer` for coordinated fields, schema validation, drafts, and multi-step transitions. Keep isolated UI state local. Trade-offs: More structure and boilerplate, but fewer invalid state combinations. Performance: Subscribe fields narrowly. Scalability: Separate local draft from submitted server data.

Dashboard filters shared by widgets

What would you do? Store shareable filters in URL state or a scoped context. Build queries from one canonical filter model. Trade-offs: URL serialization adds complexity but supports history and deep links. Performance: Avoid frequently changing broad context values. Scalability: Centralize query-key construction.

MFA authentication flow

What would you do? Separate typed code from verification mutation state. Use a reducer or state machine when states include expiry, lockout, recovery, and verification. Trade-offs: `useState` is concise initially; a reducer is safer for legal transitions. Performance: Update timers at a controlled interval. Scalability: Encapsulate transitions in a tested custom Hook.

4. Interview Red Flags

- “`setState` is asynchronous.” ■ “A setter schedules an update; React controls render and commit timing.” This is more accurate than treating it as a generic `async` API.
- “State changes immediately after `setCount`.” ■ “The current handler sees its current render snapshot.”
- “React deep-compares objects.” ■ “React uses `Object.is` for state comparisons; update objects immutably.”
- “Only click handlers batch updates.” ■ “React 18 batches many updates from promises, timers, and native events too.”
- “`useMemo` prevents rerenders.” ■ “It caches a calculation; it does not prevent its component from rendering.”
- “Use `useEffect` to sync derived state.” ■ “Derive during render unless synchronizing with an external system.”
- “All shared state must be global.” ■ “Keep state local unless multiple consumers truly require shared ownership.”

5. Frequently Asked Follow-Up Questions

- `useReducer` vs `useState`? `useReducer` fits related fields, named actions, and workflow logic.
- Can I set state during render? Generally no; it risks loops. Use events, derivation, or an effect for external synchronization.
- Can state hold functions? Yes: `useState(() => createFormatter())` stores the function/value created by the initializer.
- Does identical state always skip render? React can bail out with `Object.is`, but do not depend on exact call counts.
- When `useRef`? When a value persists without affecting UI: timers, DOM nodes, previous values.

- How reset state? Set initial values explicitly or intentionally remount with a new key.
- Can state contain Map or Set? Yes, but return a new instance after updates.
- What does Strict Mode change? Development can invoke render-related work more than once to expose impure code.

6. 30-45 Second Interview Answer

useState is React's Hook for local state in function components. It returns the current state snapshot and a setter that schedules React to render the next UI state. I use it for component-owned interaction data such as inputs, selected IDs, modal visibility, and temporary UI status. When next state depends on prior state, I use functional updates. I keep state minimal and avoid derived values, because duplicated state creates synchronization bugs. For complex workflows I use useReducer, and for API data I use a server-state layer rather than treating local state as a cache.

7. 2-3 Minute Deep Dive

useState lets a function component remember values between renders while keeping UI declarative. The key model is that state is a snapshot: a render and its handlers see one version of state. Calling a setter queues work rather than changing that render's variable. That is why cumulative updates use functional updaters.

React associates Hook state with the mounted component and Hook call order. A setter queues an update; React schedules work, renders a candidate tree, reconciles it, and commits accepted DOM changes. React 18 batches many updates automatically. Fiber supports prioritization and interruption, but Fiber queues and lanes are implementation details.

In production I use useState for local interaction concerns such as form drafts, selected rows, and open panels. I avoid using it as an API cache because remote data needs invalidation, retries, races, and sharing. I choose a reducer for complex related transitions and URL state for shareable filters.

Performance starts with state placement. Keep it close to consumers, derive values during render, virtualize large lists, and profile before adding memoization. Common mistakes are stale closures, direct repeated updates, mutating objects/Sets, assuming setters mutate synchronously, and copying derived state through effects. The interview summary is: minimal, immutable, component-owned state; explicit ownership; functional updates when prior state matters.

8. Final Revision Sheet

Final revision sheet

Definition - Hook that preserves local component state across renders.

Why - Lets React render interactive UI declaratively.

How - Returns state and setter; the setter schedules a future render.

Real-world usage - Inputs, selections, modal state, drafts, temporary status.

Performance tip - Keep state close to consumers; virtualize before over-memoizing.

Common mistake - Mutating arrays, objects, Maps, or Sets in place.

Interview trap - Repeated direct updates use the same state snapshot.

One-line definition - useState stores component-owned data and requests React to reconcile UI changes.

Memory trick - State belongs to the component instance, not the function call.